SOFTWARE ENGINEERING LAB | LAB 5

Static Code Analysis

Loan Eligibility and EMI Calculator

Student	Yash Raghavendra A.
SRN	PES2UG24CS913
Course	B.Tech Computer Science and Engineering
Lab topic	Static code analysis using Pylint, Bandit and Flake8
Source repository	SETAPESU26/L5_57_loan-eligibility
Report date	14 October 2026

Report overview

This report documents the static-analysis workflow for the assigned loan eligibility and EMI calculator: baseline inspection, issue identification, secure and maintainable code corrections, and final checks using Pylint, Bandit and Flake8.

Submission bundle: source code, issue log, reflection, and final tool reports are maintained with the lab folder in the consolidated Software Engineering repository.

1. Objective and methodology

The objective of this lab is to improve Python code quality, security and style by applying three static-analysis tools to a provided program. Static analysis inspects source code without executing it and highlights potential risks and maintainability problems.

Tools used

Tool	Purpose	Role in this lab
Pylint	Code quality and potential programming errors	Checks naming, documentation, code structure and suspicious patterns.
Bandit	Common Python security issues	Flags risky modules, credential handling and process-execution patterns.
Flake8	Style and static errors	Checks formatting, blank lines, line length and related errors.

Workflow followed

1. Setup. Created and activated a Python virtual environment and installed Pylint, Bandit and Flake8.

2. Baseline. Ran the unmodified program and saved before-fix reports for each tool.

3. Remediation. Corrected logical and numerical defects, validated inputs, improved persistence and removed flagged insecure networking/process patterns.

4. Verification. Compiled and executed the corrected program, reran the static-analysis tools, and addressed the reported style issues.

5. Documentation. Recorded issues and resolutions in issues.md and summarized the learning outcomes in reflection.md.

Key code changes

• Used 0.09 as the decimal rate for a 9% annual interest example and supported zero-interest EMI calculations.

• Validated age, income, credit score, existing EMI and allowed loan type before approving eligibility.

• Replaced the maximum-loan payment sum with a present-value calculation that accounts for interest.

• Rejected credit scores outside 300-900 and corrected risk-band validation.

• Replaced marshal-based storage with JSON and context-managed file operations.

• Moved credentials to environment variables; replaced FTP with SFTP over SSH and subprocess-based email with SMTP over TLS.

2. Issue log and corrective actions

The following items were identified by reviewing the original program and its tool findings. Tool output is specific to each tool: a code-review defect is not necessarily reported by Pylint, Bandit and Flake8 alike.

Issue	Type	Original location	Fix applied
Hard-coded FTP password	Security	`uploadToBureau()`	Credentials are read from environment variables.
FTP-related security warning	Security	Imports / upload function	Replaced FTP with SFTP over SSH using Paramiko and system host-key verification.
Subprocess-based notification	Security / portability	`notifyApplicant()`	Replaced external `mail` command with SMTP over TLS.
Unsafe marshal persistence	Security	`saveApplication()` / `loadApplications()`	Stored and loaded JSON with structural validation.
Mutable default list	Bug	`checkEligibility()`	Removed shared `history=[]` parameter.
Eligibility shortcut bypass	Logic	`checkEligibility()`	All mandatory constraints are validated before the final decision.
Incorrect annual rate example	Calculation	`main()`	Changed 9 to 0.09 for a 9% annual interest rate.
Maximum loan ignores interest	Calculation	`maxLoanAmount()`	Used the present-value formula for the permitted monthly payment.
Credit score above range accepted	Logic	`riskBand()`	Returned INVALID for scores outside 300-900.
Inaccurate prepayment comparison	Calculation	`interestSaved()`	Recalculated interest on the remaining principal before estimating savings.
Manual file handling	Maintainability	Persistence functions	Used `with open(...)` so resources are closed reliably.
Predictable identifier generator	Quality	`applicationId()`	Used the `secrets` module for identifier generation.
Style and naming issues	Style	Multiple functions	Applied `snake_case`, docstrings and PEP 8 formatting.

The issue table uses function names instead of fixed line numbers because line numbers change as the code is edited. If the instructor requires exact original line numbers, consult the saved loan_calculator_original.py from the initial lab run.

3. Verification and results

The following results were reported from the student's local PowerShell run after corrections. The program compiled and ran, and the final static-analysis checks were rerun after the formatting correction.

10.00/10	0	0 findings
Pylint score	Bandit issues reported	Flake8 final run

Functional demonstration output

```
EMI for 5L at 9% for 5 years: 10,379.18
Eligible (score 760, income 15000, EMIs 12000): False
Eligible (unknown loan type 'crypto'): False
Max loan for income 40000: 722,600.60
Last schedule row: (12, 87.97, 0.0)
Risk band for 950: INVALID
Interest saved by prepaying 50000: 12275.07
Saved application: LN-64055491
Applications on file: 4
SFTP upload and email notification skipped in demonstration.
```

Interpretation

• The 9% EMI example produces a sensible monthly payment of about 10,379.18 for a principal of 500,000 over five years.

• The eligibility function returns False for an applicant exceeding the specified EMI ratio and rejects the unsupported loan type `crypto`.

• A credit score of 950 is rejected as INVALID rather than being classified as low risk.

• The final Pylint output reported 10.00/10. The final Bandit report stated “No issues identified” with zero low, medium and high findings.

• Flake8 initially reported a missing final newline and then an E302 blank-line issue. These formatting issues were corrected; final Flake8 should be re-run on the final committed file to preserve current evidence.

The SFTP upload and SMTP notification functions were not invoked in the demonstration. They require valid server configuration and credentials; the lab run verifies the local calculator workflow, not external delivery to a real service.

4. Reflection

1. What is static code analysis, and why is it important?

Static code analysis examines source code without executing the program. It helps identify bugs, security weaknesses, maintainability problems and style violations early. Early identification can reduce debugging effort and make software easier to maintain. Static analysis supports reliable development but does not replace testing or code review.

2. What are the differences between Pylint, Bandit and Flake8?

Pylint evaluates overall Python quality, naming, documentation, structure and potential programming errors. Bandit focuses on common security risks and unsafe patterns. Flake8 checks style, whitespace, line length, unused imports and related static errors. Using all three provides broader coverage than relying on one tool.

3. What issues were identified and how were they fixed?

The original program contained a hard-coded FTP password, FTP-related security concerns, a subprocess-based notification, marshal-based persistence, a mutable default argument, an eligibility bypass, an incorrect interest-rate argument, a maximum-loan calculation that ignored interest, an invalid credit-score classification and style issues. The revised program moves credentials to environment variables, uses SFTP over SSH and SMTP over TLS, uses JSON and context managers, validates inputs and eligibility, corrects financial calculations and standardizes naming.

4. What are the limitations of static code analysis?

Static analysis cannot guarantee that software is free of all defects. Some issues depend on runtime data, configuration, intended business rules or external systems. Tools can also produce false positives or miss problems outside their rules. Static analysis should be combined with functional testing, runtime checks and human code review.

5. Submission checklist

Deliverable	Expected location	Status / note
Corrected source	`Lab05_Static_Code_Analysis/loan_calculator.py`	Included in repository folder
Issue report	`issues.md`	Included in repository folder
Reflection answers	`reflection.md`	Included in repository folder
PDF report	`Lab5_Static_Code_Analysis_Report.pdf`	Upload this PDF to the submission form
Original/final reports	`reports/` folder	Copy the authentic `.txt` reports from the local run if required

Conclusion

The exercise applied complementary static-analysis tools to improve the loan calculator. The reported final run achieved a Pylint score of 10.00/10 and no Bandit findings; after the last formatting fix, the Flake8 check should be retained with the final committed version. The source code, issue log and reflections are organized as a lab folder for inclusion in the consolidated Software Engineering repository.

Assigned lab repository: https://github.com/SETAPESU26/L5_57_loan-eligibility